Programovanie v C++: Objektovo orientovaný prístup

Prednáška 3: Užívateľom definované dátové typy (Classes & Structs)

Programovanie mechatronických aplikácií
ÚAMT FEI STU

PMEAP Prednáška 3

Motivácia: Reprezentácia komplexných dát

- Pri návrhu softvéru často narážame na potrebu reprezentovať komplexné entity.
- Príklad z geometrie: Vektor (geometrický) sa skladá z dvoch bodov – začiatočného a koncového.
- Každý z týchto bodov má svoju vlastnú x-ovú a y-ovú súradnicu.
- Ak použijeme štandardné primitívne typy, naša reprezentácia vyžaduje udržiavať štyri nezávislé premenné typu `double` (`startx`, `starty`, `endx`, `endy`).

Problém primitívnej reprezentácie

Ak vektor reprezentujeme ako sadu nezávislých premenných, kód sa stáva neprehľadným.

```
int main() {
    double xStart = 1.2;
    double xEnd = 2.0;
    double yStart = 0.4;
    double yEnd = 1.6;
    // ...
}
```

Rozbita reprezentacia

Tento prístup nemá žiadne vnútorné prepojenie. Kompilátor nevie, že tieto štyri premenné tvoria jeden logický celok.

Dôsledok: Odovzdávanie veľkého počtu parametrov

- Kvôli chýbajúcej abstrakcii musíme do funkcií odovzdávať všetky štyri premenné typu `double` samostatne.

```
1  void printVector(double x0, double x1, double y0, double y1) {
2      cout << "(" << x0 << "," << y0 << ") -> ("
3           << x1 << "," << y1 << ")" << endl;
4  }
5
6  int main() {
7      double xStart = 1.2, xEnd = 2.0, yStart = 0.4, yEnd = 1.6;
8      printVector(xStart, xEnd, yStart, yEnd);
9      // Vypise: (1.2,0.4) -> (2.0,1.6)
10 }
```

Zhoršenie problému pri modifikácii

Ak potrebujeme vektor posunúť (modifikovať), musíme navyše všetky parametre odovzdať referenciou.

```
void offsetVector(double &x0, double &x1, double &y0, double &y1,
                  double offsetX, double offsetY) {
    x0 += offsetX;
    x1 += offsetX;
    y0 += offsetY;
    y1 += offsetY;
}
```

Výsledkom je, že do funkcií sa predáva obrovské množstvo premenných. Zvyšuje sa pravdepodobnosť chyby pri volaní funkcie (napríklad zámena poradia argumentov).

Riešenie: Triedy (Classes)

Definícia triedy

Trieda (class) je užívateľom definovaný dátový typ, ktorý zoskupuje súvisiace informácie dohromady do jedného logického celku.

Týmto spôsobom vieme vytvoriť nový typ `Vector`, ktorý pod sebou zapuzdrí všetky štyri súradnice (`xStart`, `xEnd`, `yStart`, `yEnd`).

Syntaktická štruktúra triedy

Na definíciu nového dátového typu používame kľúčové slovo `class`.

```
1 class Vector {
2 public:
3     double xStart;
4     double xEnd;
5     double yStart;
6     double yEnd;
7 };
```

- Kľúčové slovo `class` a názov `Vector` určujú, že definujeme nový typ s názvom Vector.
- Definícia triedy musí byť ukončená bodkočiarkou `;`.

Polia triedy (Fields / Members)

- Premenné deklarované vo vnútri triedy nazývame polia (fields) alebo tiež členské premenné (members).
- Polia indikujú, z akých súvisiacich informácií sa náš nový dátový typ skladá.
- V inžinierskom kontexte trieda funguje ako pamäťová šablóna (blueprint). Polia určujú, koľko bajtov bude alokovaných pre každý objekt tohto typu.

Polia s rôznymi dátovými typmi

Trieda nemusí obsahovať iba polia rovnakého typu. Môže kombinovať akékoľvek známe dátové typy.

```
1 class MITStudent {
2 public:
3     char *name;
4     int studentID;
5 };
```

Tento kód definuje študenta, ktorý má vo svojom vnútri pointer na reťazec znakov (meno) a celé číslo (identifikačné číslo študenta).

Trieda vs. Inštancia

Rozdiel

Trieda je len abstraktná definícia (šablóna). Na prácu s dátami musíme vytvoriť **inštanciu** (instance).

- Inštancia predstavuje konkrétny výskyt (occurrence) danej triedy v operačnej pamäti.
- Rôzne inštancie tej istej triedy môžu mať vo svojich poliach uložené svoje vlastné, úplne nezávislé hodnoty.
- Ak chceme reprezentovať 2 rôznych študentov (s rôznymi menami a ID), vytvoríme 2 inštancie triedy `MITStudent`.

Deklarácia inštancií

Inštancie deklarujeme rovnako ako bežné premenné primitívnych typov (napr. `int x;`).

```
1 int main() {
2     // Definuje dve instancie triedy MITStudent
3     MITStudent student1;
4     MITStudent student2;
5 }
```

Pri deklarácii alokuje operačný systém na zásobníku (stack) dostatok pamäte na uloženie všetkých polí pre `student1` a separátne pre `student2`.

Prístup k poliam: Bodková notácia

Na prístup k jednotlivým poliam konkrétnej inštancie sa používa operátor prístupu (bodka) vo formáte: `premenna.nazovPola`.

```
1  int main() {
2      MITStudent student1;
3      MITStudent student2;
4
5      // Zapis do pola instancie student1
6      student1.name = "Geza";
7      student1.studentID = 123456789;
8
9      // Zapis do pola instancie student2
10     student2.name = "Jesse";
11     student2.studentID = 987654321;
12 }
```

Čítanie hodnôt z inštancií

Rovnaká bodková notácia sa používa na získavanie (čítanie) uložených hodnôt.

```
1 cout << "student1 name is " << student1.name << endl;
2 cout << "student1 id is " << student1.studentID << endl;
3
4 cout << "student2 name is " << student2.name << endl;
5 cout << "student2 id is " << student2.studentID << endl;
```

Prístup do pamäte prebieha priamo na offsete daného poľa voči počiatočnej adrese inštancie.

Abstrakcia hlbšie: Bod ako nezávislý objekt

Vráťme sa k vektoru. Vector reprezentuje dva body (začiatok a koniec). Súčasná definícia triedy `Vector` síce rieši združenie premenných, no nezobrazuje skutočnosť, že súradnice sa dajú logicky zoskupiť do ešte menšej jednotky – do **Bodu (Point)**.

Trieda Point (Bod)

Najprv si vytvoríme triedu pre samostatný Bod.

```
1 class Point {
2 public:
3     double x;
4     double y;
5 };
```

Tento prístup kopíruje skutočný fyzický a matematický model: bod pozostáva zo súradnice x a súradnice y.

Kompozícia: Polia môžu byť triedy

V C++ môžu byť polia jednej triedy inštanciami inej, už existujúcej triedy. Toto sa v softvérovom inžinierstve nazýva **kompozícia**.

```
1 class Vector {
2 public:
3     Point start;
4     Point end;
5 };
```

Trieda `Vector` teraz obsahuje dve inštancie triedy `Point` (`start` a `end`). Týmto vzniká hierarchická štruktúra dát v pamäti.

Prístup k vnoreným poliam (Chained Access)

Ak vytvoríme inštanciu vektora `vec1`, získavame prístup k jeho vnoreným vlastnostiam pomocou zreťazenej bodkovej notácie.

```
int main() {
    Vector vec1;

    // Zapis k vnorenym poliam bodu
    vec1.start.x = 3.0;
    vec1.start.y = 4.0;

    vec1.end.x = 5.0;
    vec1.end.y = 6.0;
}
```

`vec1.start.x` znamená: Pristúp do `vec1`, v ňom nájdi objekt `start`, a v ňom uprav pole `x`.

Priradenie inštancií (Assignment)

Inštancie môžeme priraďovať medzi sebou pomocou operátora =.

```
1 Vector vec1;
2 // ... inicializacia vec1 ...
3 Vector vec2;
4
5 vec2.start = vec1.start;
```

Sémantika kopírovania

Priradenie jednej inštancie druhej skopíruje kompletne všetky polia. Ide o tzv. *shallow copy* pamäťového bloku objektu.

Efekt nezávislých kópií

Po operácii priradenia vzniká nezávislá kópia. Ak následne zmeníme pole v cieľovej inštancii, originál zostáva nedotknutý.

```
1 Vector vec2;
2 vec2.start = vec1.start; // Kopiruje data
3
4 vec2.start.x = 7.0; // Meni x iba vo vec2
5 // vec1.start.x ostava povodna (napr. 3.0)
```

Odovzdávanie tried do funkcií: Hodnotou (By Value)

Základné pravidlo v C++: Objekty (inštancie tried) môžeme odovzdávať ako parametre do funkcií. Odovzdávanie hodnotou (*pass by value*) prenesie do funkcie iba **kópiu** inštancie triedy. Zmeny vykonané vnútri funkcie nie sú zachované mimo nej.

```
1  void offsetPoint(Point p, double x, double y) {
2      p.x += x; // Upravuje iba KOPIU
3      p.y += y;
4  }
5
6  int main() {
7      Point p; p.x = 3.0; p.y = 4.0;
8      offsetPoint(p, 1.0, 2.0); // Neurobi nic pre povodny objekt 'p'
9      // p ostava (3.0, 4.0)
10 }
```

Odovzdávanie tried do funkcií: Referenciou (By Reference)

Aby sa zmeny prejavili aj na pôvodnej inštancii, inštancia triedy musí byť do funkcie odovzdaná odkazom (*pass by reference*).

```
1  // Vsimnite si symbol & pri parametri p
2  void offsetPoint(Point &p, double x, double y) {
3      p.x += x; // Upravuje povodny pamatovy blok objektu
4      p.y += y;
5  }
6
7  int main() {
8      Point p; p.x = 3.0; p.y = 4.0;
9      offsetPoint(p, 1.0, 2.0); // Zmeny funguju!
10     // p je teraz (4.0, 6.0)
11 }
```

Pravidlo použitia

- **Hodnotou (By Value):** Triedy môžete odovzdať hodnotou vtedy, ak funkcia nepotrebuje modifikovať originálnu triedu (napr. funkcia len číta dáta pre výpis).
- **Referenciou (By Reference):** Triedy odovzdávajte referenciou vždy, keď ich hodnoty musia byť modifikované vo vnútri funkcie. *(V praxi sa pre efektívnosť pamäte prenášajú veľké objekty aj na čítanie pomocou tzv. const-referencie).*

Praktický návrh funkcií pre Vector

Pozrime sa, ako by teraz vyzerali naše prepracované funkcie pracujúce s triedou `Vector`.

```
1  void offsetVector(Vector &v, double offsetX, double offsetY) {
2      v.start.x += offsetX;
3      v.end.x += offsetX;
4      v.start.y += offsetY;
5      v.end.y += offsetY;
6  }
7
8  void printVector(Vector v) {
9      cout << "(" << v.start.x << "," << v.start.y << ") -> ("
10         << v.end.x << "," << v.end.y << ")" << endl;
11 }
```

Prvá funkcia objekt modifikuje (využíva referenciu `&v`), druhá iba vypisuje hodnoty (nevyžaduje referenciu).

Nedostatky voľných funkcií

Všimnite si zásadnú vec: Určité funkcie sú veľmi úzko spojené (associated) výlučne s jednou špecifickou triedou.
Funkcie `offsetVector` a `printVector` nemá zmysel volať na ničom inom, iba na inštancii typu `Vector`. Napriek tomu stoja samostatne mimo triedy. Tu prichádza koncept **Metód**.

Metódy (Methods)

Definícia metódy

Metódy sú funkcie, ktoré sú priamou súčasťou triedy.

Rozdiel oproti klasickým funkciám:

- Metódy majú automatický, implicitný prístup k poliám aktuálnej inštancie, na ktorej sú volané.
- Objektová paradigma hovorí, že inštancie si nesú nielen svoje dáta, ale aj svoje správanie.

Syntax volania metódy

Namiesto voľnej funkcie, metóda sa volá priamo "na objekte" pomocou bodkovej notácie.

```
1 Vector vec;
2 // ... inicializacia dat ...
3
4 // Volanie metody
5 vec.print();
```

V tomto prípade je `vec` naša inštancia objektu a `print` je názov vyvolanej metódy.

Vnútro metódy a implicitný kontext

- Metódy implicitne odovzdávajú aktuálnu inštanciu objektu do svojho vnútra.
- Vo vnútri metódy môžeme priamo pristupovať k poliam (`start`, `end`) bez uvádzania názvu premennej (ako by bolo nutné z vonku).

```
1  class Vector {
2  public:
3      Point start;
4      Point end;
5
6      // Metoda implementovana priamo v triede
7      void print() {
8          cout << "(" << start.x << "," << start.y << ") -> ("
9               << end.x << "," << end.y << ")" << endl;
10     }
11 };
```

Parametre v metódach

Metódam je samozrejme možné odovzdávať argumenty úplne rovnako, ako klasickým funkciám.

```
1  class Vector {
2  public:
3      Point start; Point end;
4
5      void offset(double offsetX, double offsetY) {
6          start.x += offsetX;
7          end.x += offsetX;
8          start.y += offsetY;
9          end.y += offsetY;
10     }
11 };
12
13 // Volanie:
14 vec.offset(1.0, 1.5);
```

Abstrakcia kontextu: Čie dáta sa upravujú?

Analógia: Predstavte si inštanciu triedy ako box. Metódy fungujú ako "tlačidlá" na boxe (inštancii), ktoré vykonajú určitú akciu s dátami boxu po ich stlačení.
Ak máme inštancie `vec1` a `vec2`, kompilátor presne vie, ktorú pamäť upraviť.

- Volanie `vec1.offset(...)` modifikuje výhradne polia vnútri `vec1`.
- Volanie `vec2.offset(...)` modifikuje výhradne polia vnútri `vec2`.

Technický detail: Systém C++ vkladá do každej metódy neviditeľný ukazovateľ "this", pomocou ktorého metóda presne vie, "ktoré tlačidlo bolo stlačené".

Modulárny prístup k metódam (Delegácia)

Tak ako `Vector` obsahuje polia typu `Point`, metódy jednej triedy môžu volať metódy podradených tried. Môžeme delegovať prácu.

```
1  class Point {
2  public:
3      double x, y;
4      void offset(double offsetX, double offsetY) {
5          x += offsetX; y += offsetY;
6      }
7  };
8
9  class Vector {
10 public:
11     Point start, end;
12     void offset(double offsetX, double offsetY) {
13         // Delegovanie funkcnosti na metodu triedy Point!
14         start.offset(offsetX, offsetY);
15         end.offset(offsetX, offsetY);
16     }
17 };
```

Oddelenie deklarácie od implementácie

- Pri klasických funkciách sme využívali prototypy na oznámenie deklarácie funkcie s tým, že implementácia bude nasledovať neskôr.
- Rovnako analogicky sa tento postup dá aplikovať pre metódy tried.
- Vytvoríme tzv. hlavičkový súbor (`.h`) iba s definíciou tried a deklaráciou ich metód. A implementáciu metód odsuneme do zdrojového `.cpp` súboru.

Hlavičkový súbor (vector.h)

V hlavičkovom súbore špecifikujeme iba rozhranie (interface). Telá metód chýbajú, nachádza sa tu len ich signatúra (prototyp).

```
1  // vector.h - hlavickovy subor
2  class Point {
3  public:
4      double x, y;
5      void offset(double offsetX, double offsetY);
6      void print();
7  };
8
9  class Vector {
10 public:
11     Point start, end;
12     void offset(double offsetX, double offsetY);
13     void print();
14 };
```

Zdrojový súbor s implementáciou (vector.cpp)

Pri implementácii mimo triedy musíme kompilátoru povedať, akej triede táto metóda patrí. Na to slúži Scope Resolution operátor `::`.

```
1  // vector.cpp - implementacia metod
2  #include "vector.h"
3
4  void Point::offset(double offsetX, double offsetY) {
5      x += offsetX;
6      y += offsetY;
7  }
8
9  void Vector::offset(double offsetX, double offsetY) {
10     start.offset(offsetX, offsetY);
11     end.offset(offsetX, offsetY);
12 }
```

Operátor `::` jednoznačne indikuje (určuje), ktorej konkrétnej triede je daná metóda implementovaná.

Problém s incializáciou objektov

Súčasná štruktúra vyžaduje manuálne inicializovať každé jedno pole pri vytvorení inštancie, čo môže byť zdĺhavé a náchylné na chyby (zabudnutie inicializácie).

```
1 Vector vec;
2 vec.start.x = 0.0;
3 vec.start.y = 0.0;
4 vec.end.x = 0.0;
5 vec.end.y = 0.0;
6
7 Point p;
8 p.x = 0.0;
9 p.y = 0.0;
```

Otázka znie: Môžeme inicializovať polia priamo a automaticky v momente vytvárania inštancie?.

Konštruktory (Constructors)

Odpoveďou sú konštruktory.

Definícia

Konštruktor je špeciálna metóda, ktorá je automaticky zavolaná bezprostredne vo chvíli, keď je vytvorená inštancia objektu.

Špecifiká konštruktora:

- Nemá žiadny návratový typ (ani `void`).
- Jeho názov sa musí absolútne presne zhodovať s názvom samotnej triedy.

Predvolený konštruktor (Default Constructor)

Konštruktor bez parametrov sa nazýva predvolený konštruktor. Slúži na nastavenie počiatočného bezpečného stavu objektu.

```
class Point {
public:
    double x, y;

    // Predvoleny konstruktor
    Point() {
        x = 0.0;
        y = 0.0;
        cout << "Point instance created" << endl;
    }
};

int main() {
    Point p; // Point instance created
    // p.x je automaticky 0.0, p.y je 0.0
}
```

Parametrizované konštruktory

Konštruktory môžu prijímať (akceptovať) parametre. To nám umožňuje inicializovať objekt hneď pri deklarácii vlastnými dátami.

```
1  class Point {
2  public:
3      double x, y;
4
5      // 2-parametricky konstruktor
6      Point(double nx, double ny) {
7          x = nx;
8          y = ny;
9          cout << "2-parameter constructor" << endl;
10     }
11 };
12
13 int main() {
14     Point p(2.0, 3.0); // Vola 2-parameter constructor
15     // p.x je 2.0, p.y je 3.0
16 }
```

Preťažovanie konštruktorov (Multiple Constructors)

Trieda môže mať súčasne viacero (multiple) konštruktorov s rôznym počtom a typmi argumentov (overloading).

```
1  class Point {
2  public:
3      double x, y;
4      Point() { x = 0.0; y = 0.0; }                        // Default
5      Point(double nx, double ny) { x = nx; y = ny; }   // Parameterized
6  };
7
8  int main() {
9      Point p;            // Vola defaultny konstruktor (0.0, 0.0)
10     Point q(2.0, 3.0); // Vola 2-parametricky konstruktor (2.0, 3.0)
11 }
```

Kopírovací konštruktor (Copy Constructor)

Pripomeňme si, že priradenie jednej inštancie k druhej spôsobí skopírovanie všetkých jej polí – toto využíva tzv. predvolený kopírovací konštruktor (default copy constructor).

```
1 int main() {
2     Point q(1.0, 2.0);
3     Point r = q; // Vyvolanie kopirovacieho konstruktora!
4     // r.x je 1.0, r.y je 2.0
5 }
```

Toto správanie kompilátor automaticky generuje za nás. Kopíruje bajt za bajtom.

Vlastný kopírovací konštruktor

Jazyk C++ nám umožňuje definovať vlastný (custom) kopírovací konštruktor. Prijíma referenciu na objekt rovnakého typu.

```
1  class Point {
2  public:
3      double x, y;
4
5      // Vlastny kopirovaci konstruktor
6      Point(Point &o) {
7          x = o.x;
8          y = o.y;
9          cout << "custom copy constructor" << endl;
10     }
11 };
```

Volá sa vždy pri použití kódu typu ‘Point r = q;‘. Načo je to ale dobré?

Problém plytkej kópie (Shallow Copy) s dynamickou pamäťou

Prečo by sme definovali vlastný kopírovací konštruktor? Predvolený kopírovací konštruktor (priraďujúci všetky polia 1:1) nemusí robiť to, čo v skutočnosti očakávame (a môže byť dokonca nebezpečný). Predstavte si triedu `MITStudent` s pointerom `char *name`.

```
1 MITStudent student1;
2 char n[] = "foo";
3 student1.name = n;
4
5 MITStudent student2 = student1; // Defaultna kopia!
6 student2.name[0] = 'b';
7
8 cout << student1.name; // Vypise "boo" !!!
```

Zmenou poľa name pri `student2` sme nevedome zmenili meno aj pre `student1`.

Riešenie problému: Hlboká kópia (Deep Copy)

- Pri predvolenej (plytkej) kópii sa skopírovala iba adresa smerníka `name`. Obaja študenti fyzicky zdieľali jeden pamäťový blok pre meno.
- Vlastným kopírovacím konštruktorom vieme naimplementovať hlbokú kópiu (Deep copy).
- Pri hlbokej kópii naalokujeme nový nezávislý pamäťový blok pre `name` cieľového objektu a znaky prekopírujeme manuálne (napríklad funkciou `strdup()`).

Implementácia hlbokej kópie

```
1  class MITStudent {
2  public:
3      int studentID; char *name;
4
5      MITStudent(MITStudent &o) {
6          studentID = o.studentID;
7          name = strdup(o.name); // Skopiruje hodnotu do Novej pamate
8      }
9  };
10
11 int main() {
12     MITStudent student1; student1.name = "foo";
13     MITStudent student2 = student1; // Vola custom copy constructor
14     student2.name[0] = 'b';
15     cout << student1.name; // Bezpecne vypise "foo"
16 }
```

Zmenou študenta 2 už neovplyvňujeme študenta 1.

Zapuzdrenie (Encapsulation)

V objektovom programovaní platí silný inžiniersky princíp – interný stav objektov by nemal byť bezprostredne dostupný a meniteľný kýmkoľvek zvonku.
Tento prístup bráni neoprávnenému zápisu (napr. zadaniu záporného polomeru pre kruh). Dosahuje sa to mechanizmom kontroly prístupu.

Modifikátory prístupu (Access Modifiers)

Modifikátory prístupu v C++ definujú, odkiaľ a akým spôsobom môžu byť vaše polia a metódy sprístupnené.
Dva primárne modifikátory:

- `public`**:** Definuje verejné rozhranie. Členy sú prístupné odkiaľkoľvek.
- `private`**:** Definuje súkromnú oblasť. Členy môžu byť modifikované a čítané **výlučne iba priamo z vnútra inštancie danej triedy**.

Použitie modifikátora public

Členy v sekcii `public:` sú dostupné mimo triedy z iných funkcií.

```
1  class Point {
2  public:
3      double x, y;
4      Point(double nx, double ny) { x = nx; y = ny; }
5  };
6
7  int main() {
8      Point p(2.0, 3.0);
9      p.x = 5.0; // Povolene (allowed), 'x' je public
10 }
```

Použitie modifikátora private

Členy v sekcii `private:` sú mimo triedy skryté.

```
1  class Point {
2  private:
3      double x, y;
4  public:
5      Point(double nx, double ny) { x = nx; y = ny; }
6  };
7
8  int main() {
9      Point p(2.0, 3.0);
10     p.x = 5.0; // ERROR! Nepovolene (not allowed), 'x' je private
11 }
```

Týmto spôsobom sme uzamkli súradnice pred cudzím vonkajším zásahom.

Getters a Setters

Ak máme súkromné polia, ale chceme užívateľom povoliť čítanie hodnôt bez rizika ich zmeny z vonku, použijeme metódy známe ako tzv. "Getters". Gettery umožňujú iba *read-only* (iba na čítanie) prístup k privátnym (private) poliam.

```
1  class Point {
2  private:
3      double x, y;
4  public:
5      Point(double nx, double ny) { x = nx; y = ny; }
6
7      double getX() { return x; } // Getter
8      double getY() { return y; } // Getter
9  };
10
11 int main() {
12     Point p(2.0, 3.0);
13     cout << p.getX() << endl; // Dovolene pristupenie k hodnote
14 }
```

Predvolené pravidlá (Default Access Modifiers)

Je kritické poznať pravidlá kompilátora, ak neuvediete žiadny prístupový modifikátor explicitne.

Pravidlo Class

V konštrukcii `class` je všetko štandardne (by default) **private**.

```
1 class Point {
2     double x, y; // Toto je automaticky PRIVATE
3 };
4
5 // ... je 100% ekvivalentne zapisu:
6 class Point {
7 private:
8     double x, y;
9 };
```

Dátová štruktúra struct

Štruktúry (`struct`) sú historickým dedičstvom a prenosom z jazyka C. V jazyku C++ sa namiesto nich dnes vo všeobecnosti preferujú priamo triedy (`classes`).
V prostredí C++ sú však štruktúry v podstate úplne to isté (rovnaké funkcionality, metódy, konštruktory) ako triedy.

Zásadný rozdiel struct vs class

Aké je rozlíšenie? Prečo existujú obe? Zásadný rozdiel je jedine v **predvolenom prístupovom modifikátore (default access modifier)**.

Pravidlo Struct

V konštrukcii `struct` je všetko štandardne (by default) **public**. Zatiaľ čo u tried to bolo private.

Ukážka ekvivalencie Struct

```
1 struct Point {
2     double x, y; // Automaticky PUBLIC
3 };
4
5 // ... je 100% ekvivalentne zapisu:
6 struct Point {
7 public:
8     double x, y;
9 };
```

Z tohto dôvodu vývojári C++ často vyhradzujú použitie `struct` iba pre úplne jednoduché prepravky dát (tzv. Plain Old Data - POD), kde nie sú potrebné privátne obmedzenia. Na zložitejšie objekty s logikou a kontrolou prístupu používajú vždy `class`.

Záverečné zhrnutie pre inžinierov

- **Triedy (Classes)** prinášajú abstraktnú silu na vytváranie vlastných logických celkov (Typov).
- Polia **kompozície** budujú rozsiahle hierarchie.
- **Referenčné odovzdávanie** optimalizuje prístup k pamäti.
- **Metódy a objektová orientácia** združujú dáta so správaním, ktoré k nim prislúcha (kontext ‘this‘).
- **Konštruktory** garantujú bezpečný a definovaný vznik inštancií v pamäti.
- **Modifikátory prístupu (Encapsulation)** zamedzujú nekontrolovaným vonkajším zásahom do štruktúry.

Q & A

Ďakujem za pozornosť!